Project Defense Drill

Gridiron Atlas - a focused 3-5 minute runbook and speaking guide

One-sentence architecture

Gridiron Atlas is an Astro 7 server-rendered NFL dashboard packaged with the Node standalone adapter in a Docker image, deployed as an ECS Fargate service behind an Application Load Balancer and CloudFront.

Your 30-second opening

“The application is an SSR Astro service because it needs same-origin dynamic API routes and server-side provider normalization. The load balancer checks **/api/ready**, the container checks **/api/health**, and ECS uses a rolling deployment with a circuit breaker. That lets a healthy old task continue serving traffic while a new task proves readiness.”

Defense sequence

Time	Say / show
0:00	Show the deployed app and identify the GitHub PR / commit that introduced the fault.
0:30	Open GitHub Actions or CloudWatch logs. State the failing path, status, and source line; do not guess.
1:00	Show that /api/health and /api/ready are 200. Explain that this contains the user-facing fault without taking the service out of rotation.
1:30	Merge the rollback PR (or show its completed deploy). Verify the bad URL is gone and health remains 200.
2:30	Defend SSR, health probes, rolling deployment, SLI/SLO, and circuit breaker choices.

Fault and rollback evidence

Planned fault

The demonstration release adds an NFL / CFB switcher. Selecting CFB reaches a provider path that has not been configured and returns an application-level 500. The ordinary NFL path remains available. This is intentionally narrow: it demonstrates a real bad release signal without making the platform unavailable.

What to show in logs

Filter the ECS CloudWatch log stream by the request ID or the CFB request. The application emits structured JSON through Astro middleware. The expected record is **path: /**, **status: 500**, and **error: Error**; the stack trace identifies **src/pages/index.astro:30** with “CFB data provider is not configured.” Explain that this is the root cause, not merely a symptom.

Proof of availability

The local rehearsal produced 200 from **/api/ready** and **/api/health**, while **/?sport=cfb** produced 500. In production, re-run those requests before and after rollback. A successful readiness response is the deployment gate; it does not mean every feature path is correct, which is why the error logs and release verification matter.

Useful commands

```
curl -i https://gridironatlas.app/api/ready curl -i https://gridironatlas.app/api/health curl -i 'https://gridironatlas.app/?sport=cfb' gh run list --workflow 'Deploy Astro SSR Container via AWS CDK' --limit 5
```

Rollback story

The rollback is a small, separate PR that removes the selector and the failing CFB path. Merging it triggers the same CI and CDK pipeline. ECS starts replacement tasks, waits for the ALB readiness check at **/api/ready**, keeps the old task healthy during the transition (minimum healthy 100%, maximum healthy 200%), then removes the old task. ECS deployment circuit breaker is enabled, so a failed replacement reverts automatically rather than draining the healthy version.

Architecture and DevOps answers

Why SSR rather than static SSG?

Static S3/CloudFront is excellent for immutable pages: lower operational cost, simple caching, and no server runtime. This project needs dynamic same-origin endpoints such as health, readiness, metrics, live data, and provider-backed pages. Containerized SSR keeps provider credentials and normalization server-side, enables operational endpoints, and supports dynamic data. The trade-off is container, load-balancer, observability, and deployment complexity.

What are the health probes?

The Docker container health check calls **http://127.0.0.1:4321/api/health** every 30 seconds with a 5-second timeout, 3 retries, and a 30-second start period. The ALB target group checks **/api/ready** and only routes to HTTP 200 targets. Liveness answers “is the process alive?”; readiness answers “should this task receive traffic?”

How do you achieve no/minimal downtime?

ECS rolling deployment is configured with **minHealthyPercent: 100** and **maxHealthyPercent: 200**. ECS can start a replacement before stopping the previous task, and the ALB routes only after readiness succeeds. With desired count 1, this is the critical configuration that avoids a planned gap during a healthy deployment. It is still honest to say no system can promise literal zero interruption for every failure mode; the evidence is continuous 200 health/readiness checks and deployment events.

SLI, SLO, and error budget

An SLI is a measured indicator, for example successful ALB requests / total ALB requests, or latency at p95. An SLO is the target, such as 99.9% successful requests over 30 days or p95 below an agreed threshold. An error budget is the allowed failure portion: at 99.9%, the budget is 0.1%. This project exposes request/error/latency metrics, has CloudWatch alarms for target 5xx, error logs, CPU, and memory, and records structured request logs.

Why not simply hot-fix production?

A PR-driven rollback leaves an auditable commit, code review record, CI result, deployment record, and a clear before/after narrative. If immediate service restoration were required, rolling ECS back to the prior known-good task definition/image is faster; follow with a source-control revert so deployed state and repository history match.

SSL / browser trust setup

Use AWS Certificate Manager (ACM), not a certificate file bought from a registrar. This project already declares a DNS-validated ACM certificate in **us-east-1** for **gridironatlas.app** and **www.gridironatlas.app**; CloudFront presents it to visitors. Deploy **GridironAtlasCertificateStack**, then copy every ACM validation CNAME exactly into Name.com. Do not delete those CNAMEs: ACM uses them for automatic renewal. Wait until ACM says **ISSUED**, deploy **CdkStack**, then point Name.com ANAME records for @ and **www** to the returned CloudFront domain. Never point the public domain at the ECS load balancer. After DNS propagates, check both HTTPS health URLs; the browser should show a valid lock when the hostname, CloudFront alternate domain, and ACM certificate all match.

```
cd cdk && npx cdk deploy GridironAtlasCertificateStack --require-approval never aws acm
describe-certificate --region us-east-1 --certificate-arn --query 'Certificate.Status' --output
text # At Name.com: add ACM CNAME validation records, then ANAME @ and www -> CloudFrontDomainName
curl --fail --location https://gridironatlas.app/api/health curl --fail --location
https://www.gridironatlas.app/api/health
```

Pre-defense checklist

1. Have PR links, commit SHAs, GitHub Actions, and CloudWatch open in tabs. 2. Know the public URL and the two health URLs. 3. Keep one terminal ready for curl commands. 4. State the exact failing line before proposing rollback. 5. After rollback deployment, repeat the three requests and capture the 200 evidence. 6. Be precise: this is a contained user-path failure, while platform health stayed available.